STUDENT MANAGEMENT SYSTEM

DBMS Mini Project Report

Submitted in partial fulfilment of the requirements for the
Database Management Systems (DBMS) Course

Project Title	Student Management System
Database Used	PostgreSQL
Backend	Node.js, Express.js
Frontend	HTML, CSS, JavaScript
Submitted By	DS-03, 18, 48, 51

Academic Year 2025 – 2026

Table of Contents

1. Introduction

The **Student Management System (SMS)** is a database-driven web application designed to digitize and simplify the process of storing, managing, and retrieving student records inside an academic institution. Traditionally, student data such as name, roll number, department, semester and email is maintained manually in registers or scattered spreadsheets, which is time-consuming, error-prone, and difficult to search or update.

This project was developed as a mini project for the Database Management Systems (DBMS) course, with the primary goal of demonstrating practical understanding of relational database design, SQL query writing, and full-stack web development by connecting a PostgreSQL database to a Node.js/Express backend and a simple HTML/CSS/JavaScript frontend.

The application allows an administrator to add new student records through a web form, and the data is validated and stored directly into a PostgreSQL relational database, which forms the single source of truth for all student information.

2. Objectives

- To design a normalized relational database schema for storing student records.
- To implement CRUD (Create, Read, Update, Delete) style operations on student data using SQL.
- To build a RESTful backend API using Node.js and Express.js that communicates with PostgreSQL.
- To design a simple, responsive front-end form for data entry and display.
- To apply DBMS concepts such as primary keys, unique constraints, and default values in a real project.
- To gain hands-on experience with tools such as pgAdmin 4 and Visual Studio Code.

3. Scope of the Project

The current scope of the system covers the **Student** entity only. It allows an administrator to add a new student's Name, Roll Number, Department, Semester and Email through a web-based form, and the record is stored permanently in a PostgreSQL database table named `students`. The system ensures that each Roll Number is unique across all records and automatically records the timestamp of when a student entry was created.

The project is built with future extensibility in mind — the same architecture (Express routes + PostgreSQL) can later be expanded to include additional modules such as Courses, Enrollments, Attendance and a Dashboard, which are discussed further in the Future Scope section.

4. System Requirements

4.1 Software Requirements

- Operating System : Windows 10 / 11
- Database Server : PostgreSQL 18
- Database Admin Tool : pgAdmin 4
- Backend Runtime : Node.js (with Express.js and pg packages)
- Code Editor : Visual Studio Code
- Browser : Google Chrome / Microsoft Edge

4.2 Hardware Requirements

- Processor : Dual-core or higher
- RAM : Minimum 4 GB
- Storage : 500 MB free disk space

5. Technology Stack

Layer	Technology Used
Database	PostgreSQL
Backend Server	Node.js + Express.js
Database Driver	pg (node-postgres)
Frontend	HTML5, CSS3, JavaScript (Vanilla)
Cross-Origin Support	CORS middleware
Database Admin Tool	pgAdmin 4

6. System Design

6.1 Entity-Relationship (ER) Diagram

Since the current implementation revolves around a single core entity, the ER diagram below represents the **STUDENTS** entity using Chen notation, with its attributes shown as ovals. The **id** attribute (underlined) is the Primary Key, and **roll_no** carries a Unique constraint.

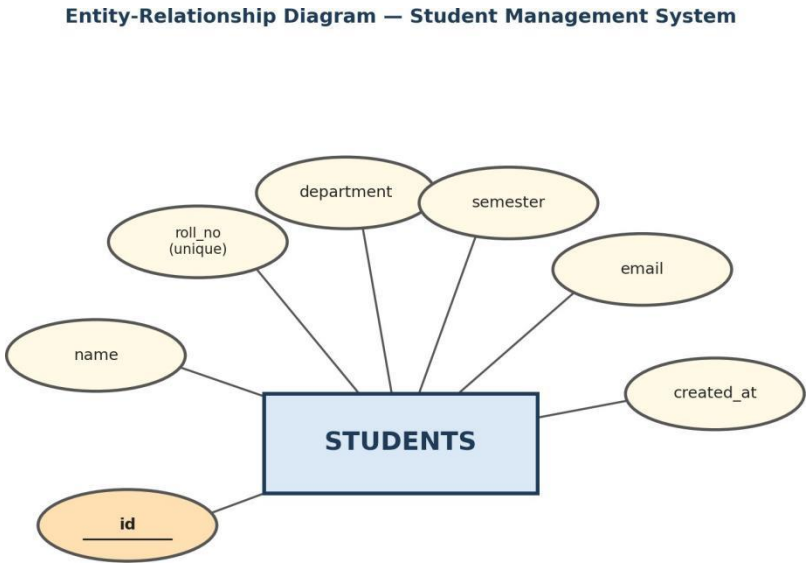


Fig 6.1 — ER Diagram of the Student Management System

6.2 Database Schema

The database consists of a single table, **`students`**, created inside the PostgreSQL database `student_management`. Its structure is as follows:

Column	Data Type	Constraint	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing unique student ID
name	VARCHAR(100)	NOT NULL	Full name of the student
roll_no	VARCHAR(50)	UNIQUE, NOT NULL	Unique roll number of the student
department	VARCHAR(100)	NOT NULL	Department / branch of the student
semester	INT	NOT NULL	Current semester of the student
email	VARCHAR(100)	—	Email address of the student
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Fig 6.2 — pgAdmin 4 view showing the schema creation query and sample data

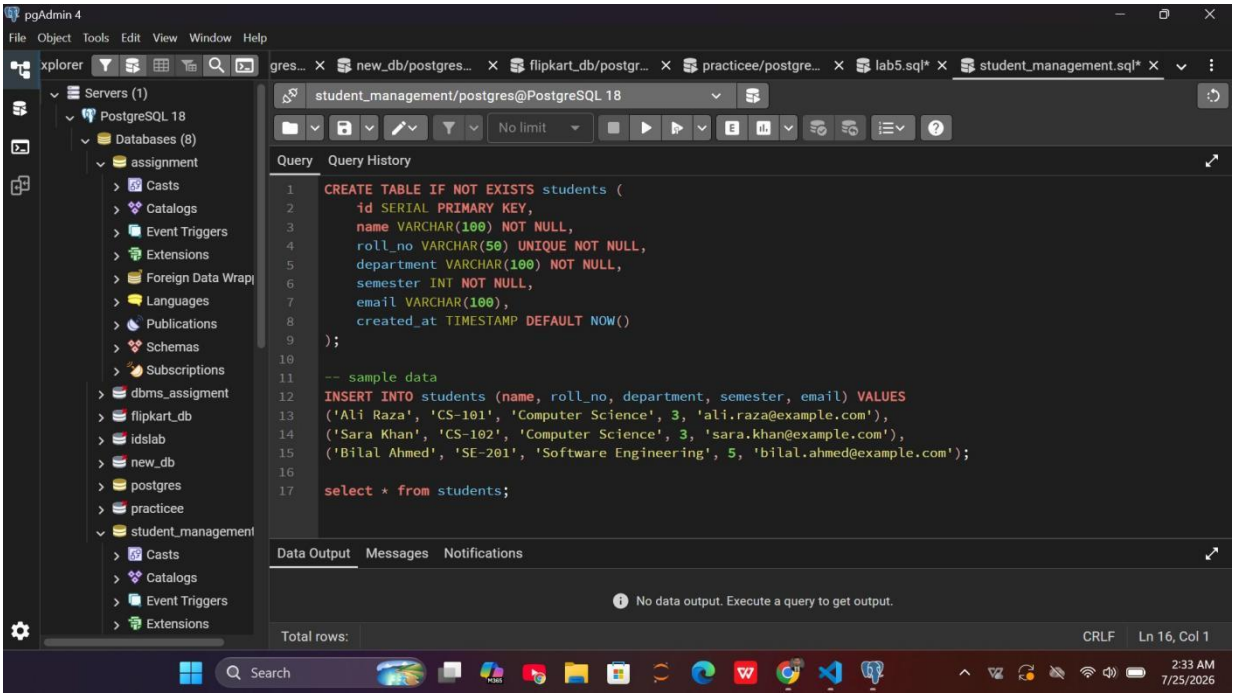


Fig 6.3 — CREATE TABLE query executed successfully in pgAdmin 4

7. Implementation

7.1 Backend (Node.js + Express + PostgreSQL)

The backend is built using **Node.js** with the **Express.js** framework. It uses the `pg` package to connect to the PostgreSQL database through a connection pool. The server exposes REST API endpoints (e.g. `GET /api/students`) that the frontend calls using JavaScript's Fetch API to retrieve or insert student records. CORS middleware is enabled to allow the frontend (served as static files) to communicate with the API, and the server listens on port 3000.

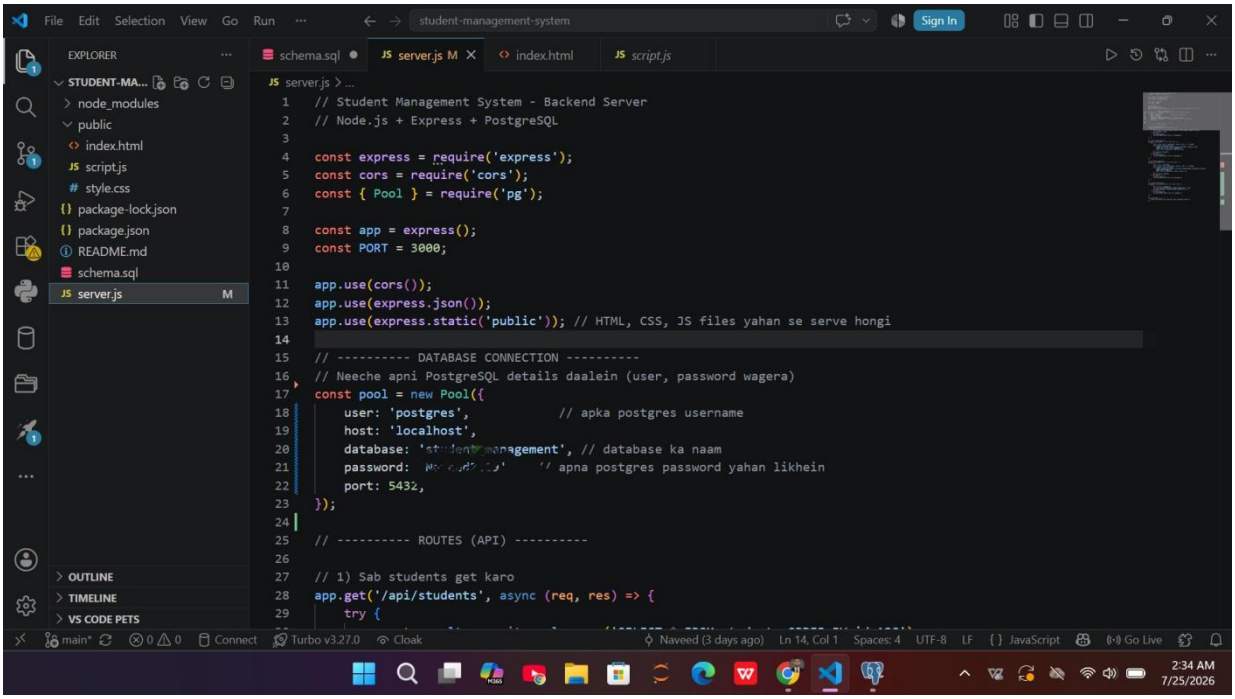


Fig 7.1 — server.js: Express server setup and PostgreSQL connection pool (VS Code)

7.2 Frontend (HTML / CSS / JavaScript)

The frontend is a simple static web page (`index.html`) styled with `style.css` and made interactive using `script.js`. It presents an "Add New Student" form with fields for Name, Roll No, Department, Semester and Email. On submission, the form data is sent to the backend API using JavaScript's Fetch API, and the new record is inserted into the database.

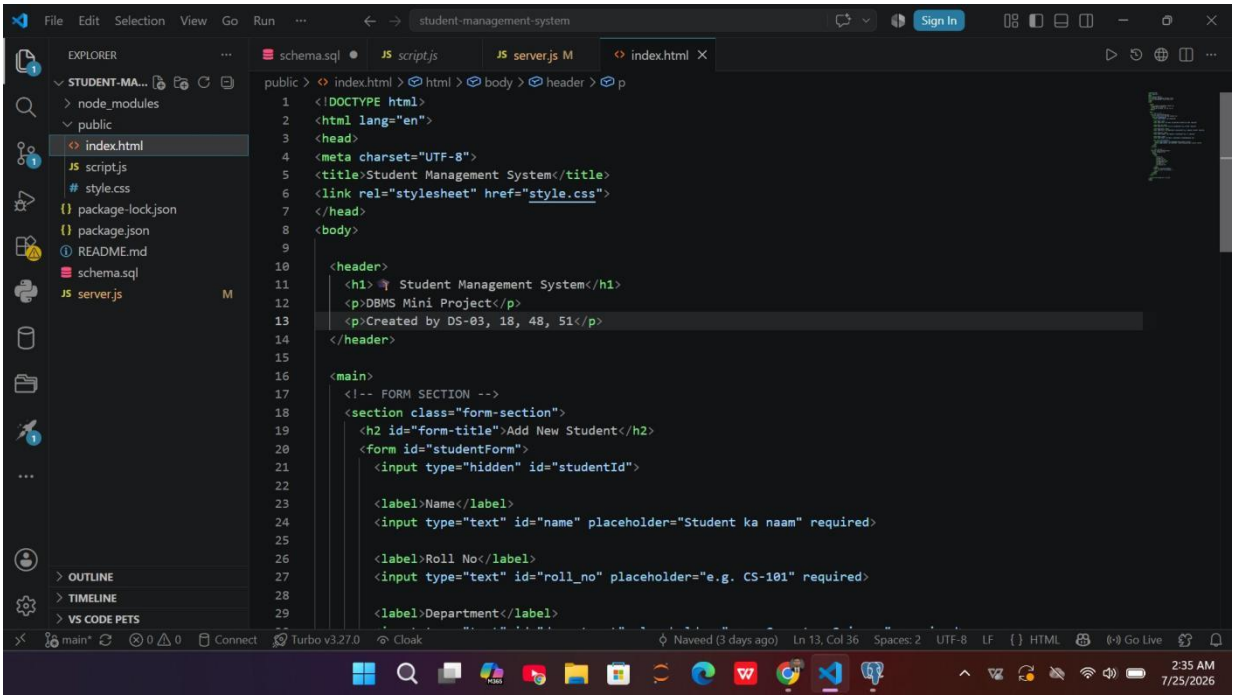


Fig 7.2 — index.html: structure of the student registration form (VS Code)

8. SQL Queries Used

Below are the core SQL queries used to design and populate the database, executed and verified using pgAdmin 4's Query Tool.

8.1 Table Creation

```
CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    roll_no VARCHAR(50) UNIQUE NOT NULL,
    department VARCHAR(100) NOT NULL,
    semester INT NOT NULL,
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT NOW()
);
```

8.2 Inserting Sample Records

```
INSERT INTO students (name, roll_no, department, semester, email)
VALUES
('Ali Raza', 'CS-101', 'Computer Science', 3, 'ali.raza@example.com'),
('Sara Khan', 'CS-102', 'Computer Science', 3, 'sara.khan@example.com'),
('Bilal Ahmed', 'SE-201', 'Software Engineering', 5, 'bilal.ahmed@example.com');
```

8.3 Retrieving All Records

```
SELECT * FROM students;
```

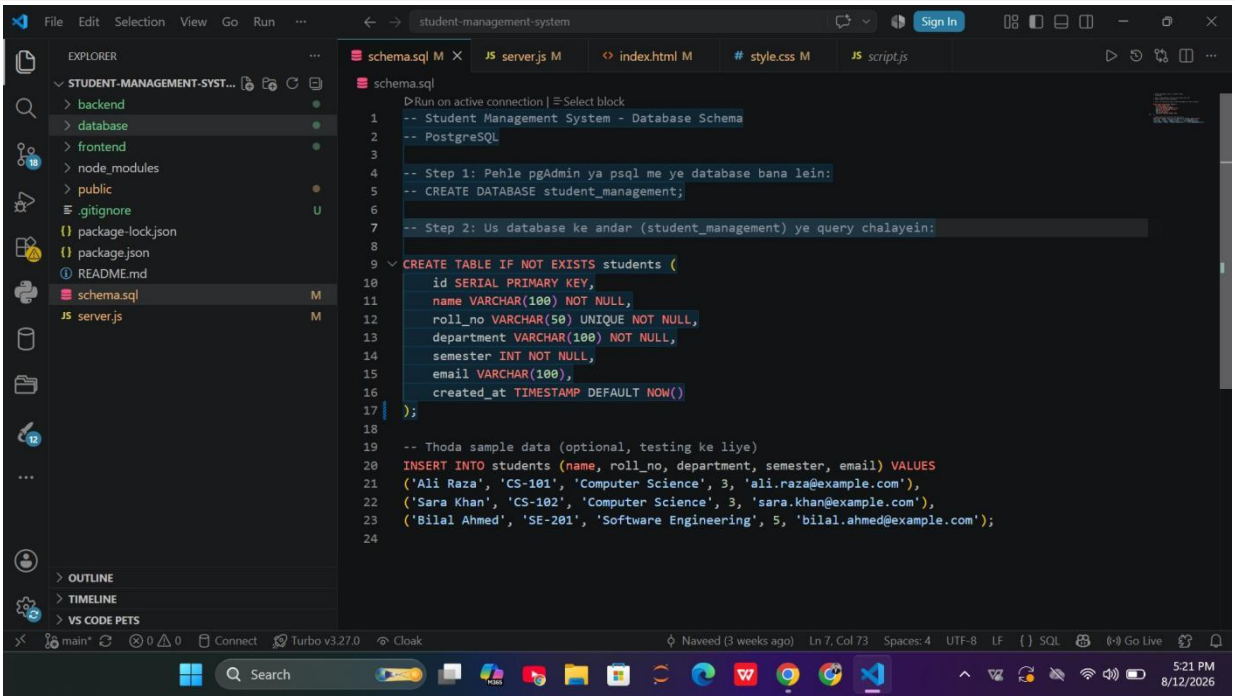
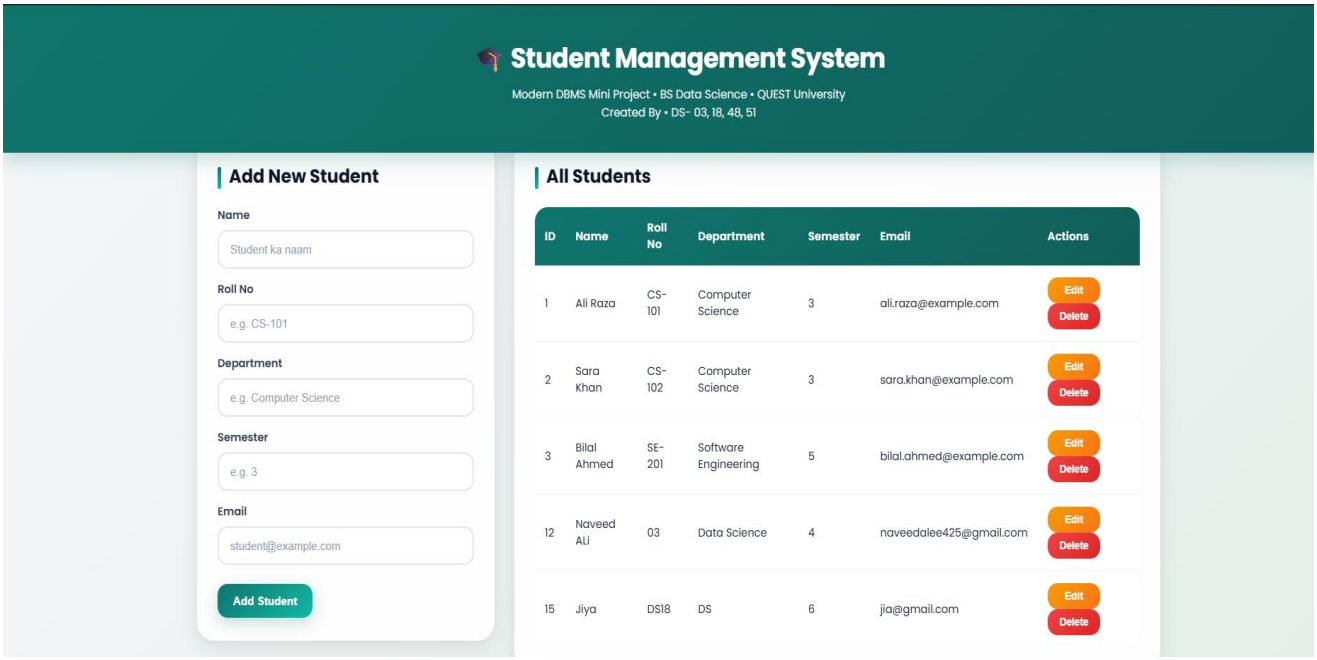


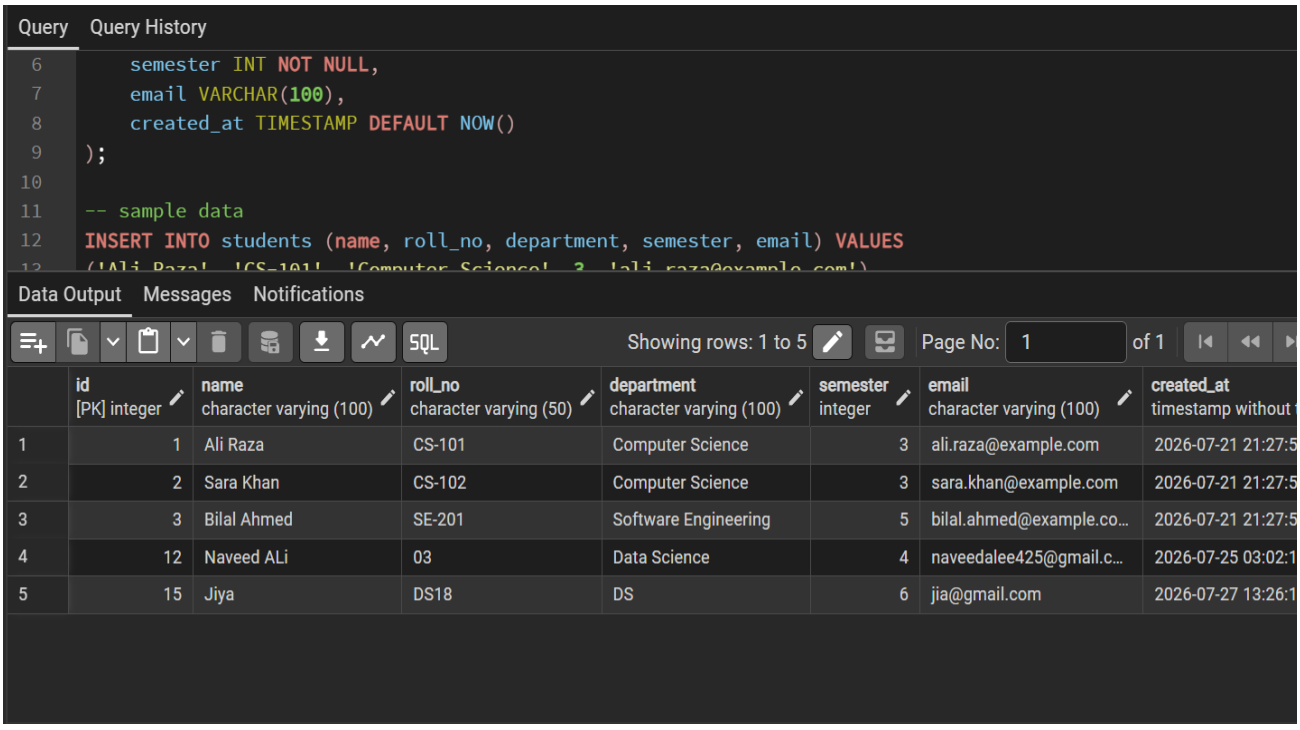
Fig 8.1 — schema.sql opened in VS Code, showing the complete DDL and DML script

9. Output / Screenshots

The following screenshot shows the running web application's "Add New Student" form, where a user can enter student details such as Name, Roll No, Department and Semester before submitting the record to the database.

Fig 9.1 — Student Management System web interface (Add New Student form)





10. Advantages of the System

- Centralized storage of student data in a structured relational database.
- Prevents duplicate entries through the UNIQUE constraint on roll_no.
- Fast and reliable data retrieval using indexed primary key lookups.
- Clean separation of concerns between frontend, backend and database layers.
- Automatic timestamping of records using the created_at column.
- Simple, user-friendly form-based interface requiring no technical knowledge to operate.

11. Limitations

- Currently supports only the Student entity; no linked Courses or Attendance data yet.
- No user authentication / login system is implemented for the admin panel.
- No update or delete functionality is exposed on the current frontend form.
- Application has not yet been deployed to a live/public server; it currently runs locally.

12. Future Scope

The project's backend already anticipates future growth, with route files reserved for `courses`, `enrollments`, `attendance` and a `dashboard`. Planned enhancements include:

- Adding Courses and Enrollments tables with foreign-key relationships to Students.
- Implementing an Attendance tracking module linked to enrollments.
- Adding Edit and Delete functionality for full CRUD support.
- Introducing admin login/authentication for secure access.
- Deploying the application to a cloud platform for public access.
- Building an analytics dashboard summarizing department-wise and semester-wise student counts.

13. Conclusion

The Student Management System successfully demonstrates the practical application of core DBMS concepts — table design, constraints, and SQL querying — integrated with a modern full-stack web architecture using Node.js, Express.js and PostgreSQL. The project provided hands-on experience in designing a relational schema, writing and testing SQL queries in pgAdmin 4, and connecting a database to a working web application. It lays a solid, extensible foundation that can be built upon to create a complete institutional student record management platform.

14. References

- PostgreSQL Official Documentation — https://www.postgresql.org/docs/
- Express.js Official Documentation — https://expressjs.com/
- Node.js Official Documentation — https://nodejs.org/en/docs/
- node-postgres (pg) Documentation — https://node-postgres.com/
- pgAdmin 4 Documentation — https://www.pgadmin.org/docs/